SRS File

1. Introduction

1.1 Background

Rapid urbanization, industrialization, and vehicular growth have led to severe air pollution, posing a critical threat to public health and global ecosystems. Particulate matter (PM_{2.5}, PM_{10}), gaseous pollutants (NO_2, SO_2, CO, O_3), and changing meteorological factors severely impact human respiratory and cardiovascular systems. To mitigate these risks, accurate monitoring and forecasting of the **Air Quality Index (AQI)** are crucial.

While traditional monitoring systems report historical pollution data, they often fail to provide proactive insights. Modern advancements in **Artificial Intelligence (AI)** and **Machine Learning (ML)** enable precise predictive modeling using public sensor data. However, many advanced deep learning models operate as "black boxes," making it difficult for policymakers and the public to trust or understand *why* a specific high-pollution alert was triggered.

1.2 Problem Statement

Despite the availability of vast public sensor data (via IoT networks, OpenAQ, and government monitoring stations), two major challenges persist:

1. **Lack of Proactive Forecasting:** Most platforms only show current pollution levels rather than forecasting future spikes, limiting preventive public health measures.
2. **The "Black Box" Problem in AI:** Advanced prediction models (like LSTMs or Gradient Boosting) lack interpretability. Without understanding the driving factors (e.g., whether a spike is caused by stubble burning, vehicular traffic, or adverse weather), municipal authorities struggle to design targeted interventions.

1.3 Objectives

The primary objectives of this project are:

- **Data Ingestion & Integration:** To collect, clean, and preprocess real-time public environmental and meteorological data from open-source APIs.
- **Predictive Modeling:** To develop and train machine learning/deep learning models (such as Random Forest, XGBoost, or LSTM) to predict short- and medium-term AQI values accurately.
- **Explainable AI (XAI) Integration:** To incorporate interpretation frameworks like **SHAP (SHapley Additive exPlanations)** or **LIME** to break down the black-box nature of the models and visualize feature contributions for every prediction.
- **Interactive Visualization:** To build a user-friendly, web-based dashboard that displays live tracking, forecasts, and transparent explanations along with automated health advisories.

1.4 Significance and Scope

This project bridges the gap between predictive analytics and human interpretability in environmental science. By making AI predictions **transparent and explainable**, it empowers:

- **Citizens:** To make informed daily decisions regarding outdoor activities based on localized health risks.
- **Policymakers & Municipal Authorities:** To pinpoint exact pollution drivers and implement precise, data-driven mitigation policies (e.g., traffic restrictions or industrial curbs).

2. Overall Description

2.1 Product Perspective

The system functions as an end-to-end web-based environmental analytics platform designed following a decoupled client-server architecture. It integrates public environmental data streams with advanced machine learning and explainable AI frameworks.

The system relies on a three-tier architecture:

- **Presentation Tier:** A responsive React-based Web UI providing intuitive dashboards, dynamic charts, and interactive controls for users.
- **Application/Logic Tier:** A Python-based **Flask Backend** that handles REST API requests, data preprocessing, machine learning inferences, and SHAP/LIME-based explainability computations.
- **Data Persistence Tier:** A **MySQL Database** responsible for securely storing user profiles, historical sensor data, prediction outcomes, and generated logs.

2.2 Product Functions

The core capabilities and functionalities delivered by the platform include:

- **Data Ingestion & Management:** Ability to process real-time or bulk uploaded CSV sensor datasets containing meteorological and pollutant attributes (PM_{2.5}, PM_{10}, NO_2, SO_2, CO, O_3, temperature, humidity).
- **Automated AQI Prediction:** Execution of trained machine learning models (e.g., Random Forest or XGBoost) to calculate numerical AQI scores and map them to standard regulatory air quality color categories.
- **Explainable Environmental Analytics (XAI):** Integration of **SHAP** feature attribution algorithms to explicitly display *why* a specific prediction was made (e.g., highlighting whether high PM_{2.5} or adverse humidity was the primary driver).
- **Interactive Visualization & Reporting:** Real-time trend graphs (daily, weekly, monthly) and the facility to download comprehensive analytical PDF reports.

2.3 User Classes and Characteristics

- **General Citizens:** Non-technical users who require quick, color-coded health status assessments, current pollution levels, and automated safety advisories.
- **Environmental Researchers:** Users who need deep data insights, correlation metrics, trend analyses, and historical records for study purposes.

- **Policymakers / Municipal Authorities:** Decision-makers who require explainable analytics and hotspot identification to formulate targeted emission control policies.

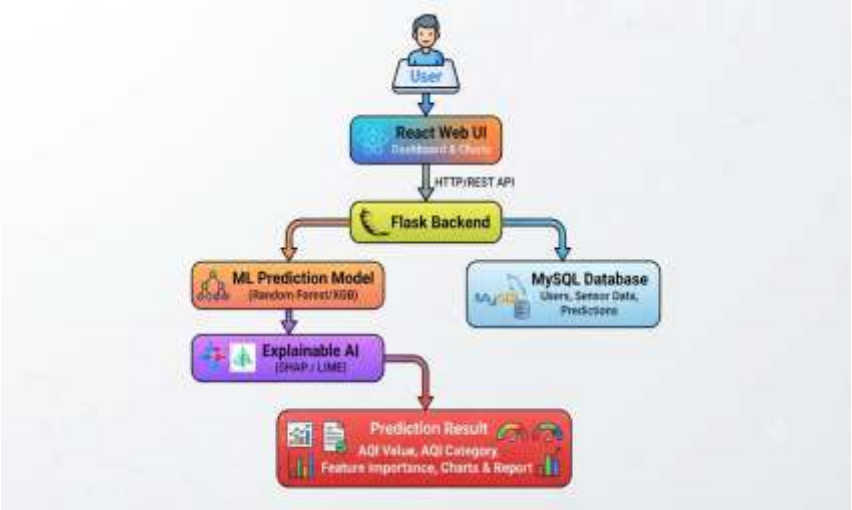
2.4 Operating Environment

- **Client Side:** Modern web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) running on desktops, tablets, or mobile devices.
- **Server Side:** Python runtime environment hosting the Flask backend API, connected to a MySQL database server.

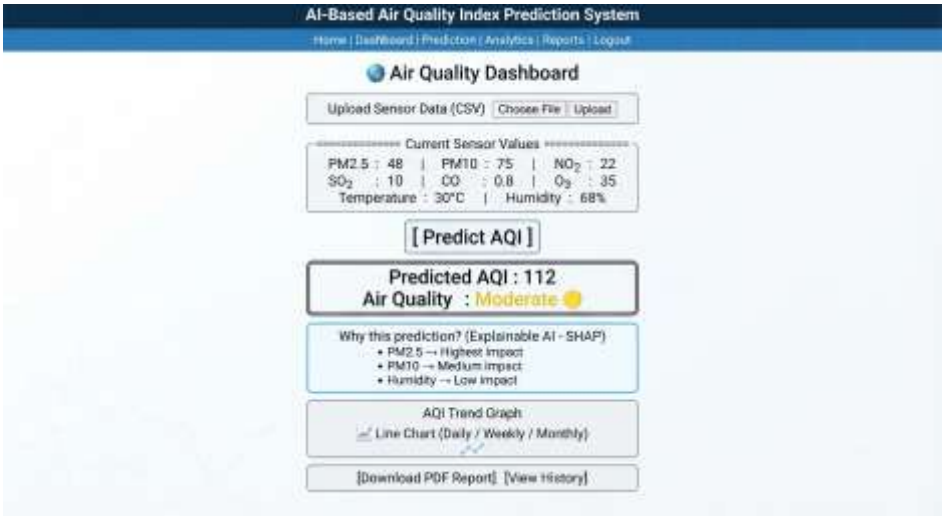
2.5 Design and Implementation Constraints

- Dependency on the availability and accuracy of public sensor data sources or pre-formatted input datasets.
- Real-time machine learning inference speed is bound by backend server processing constraints.

- **System Architecture Flow**



- **Dashboard User Interface Mockup**



- **<u>Complete Project Directory Structure</u>**

```
AI_AQI_Prediction_System/
│
├── frontend/                  # React Application
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Prediction.jsx
│   │   │   ├── Analytics.jsx
│   │   │   ├── Report.jsx
│   │   │   └── Charts.jsx
│   │   ├── pages/
│   │   ├── assets/
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
│
├── backend/                   # Flask Backend
│   ├── app.py
│   ├── routes.py
│   ├── model.py
│   ├── preprocess.py
│   ├── explain.py
│   ├── database.py
│   ├── requirements.txt
│   └── models/
```

```
│       └── aqi_model.pkl
│
├── dataset/
│   ├── air_quality.csv
│   └── cleaned_data.csv
│
├── database/
│   └── schema.sql
│
├── reports/
│   └── generated_reports.pdf
│
├── diagrams/
│   ├── UseCase.png
│   ├── ClassDiagram.png
│   ├── SequenceDiagram.png
│   ├── ActivityDiagram.png
│   └── Architecture.png
│
├── documentation/
│   ├── SRS.pdf
│   ├── Project_Report.pdf
│   └── PPT.pptx
│
├── README.md
└── .gitignore
```

3. Specific Requirements

3.1 Functional Requirements

FR-1: User Authentication & Role Management

- **FR-1.1:** The system shall allow users (Citizens, Researchers, and Policymakers) to securely log in or register into the platform.
- **FR-1.2:** The system shall restrict administrative tasks (such as dataset updates or system configuration) to authorized personnel via Role-Based Access Control (RBAC).

FR-2: Data Ingestion and Preprocessing Module

- **FR-2.1:** The system shall provide an interface for users to upload historical or real-time sensor data files in CSV format via the web dashboard.
- **FR-2.2:** The backend shall automatically parse incoming datasets and handle missing values through automated imputation techniques (e.g., mean/median or forward-fill).
- **FR-2.3:** The preprocessing engine shall normalize numerical features (such as $PM_{2.5}, PM_{10}, NO_2, SO_2, CO, O_3$, temperature, and humidity) before feeding them into the machine learning model.

FR-3: AI-Based AQI Prediction Module

- **FR-3.1:** The system shall load pre-trained machine learning/ensemble models (e.g., Random Forest or XGBoost) to process input sensor parameters.
- **FR-3.2:** The system shall compute a precise numerical Air Quality Index (AQI) value based on the processed feature inputs.
- **FR-3.3:** The system shall map the calculated numerical AQI score to standardized health categories and color codes (e.g., Good $\rightarrow$ Green, Moderate $\rightarrow$ Yellow, Unhealthy $\rightarrow$ Red).

FR-4: Explainable Environmental Analytics Module (XAI)

- **FR-4.1:** For every prediction generated, the backend shall invoke the SHAP (SHapley Additive exPlanations) or LIME module to evaluate feature attribution.
- **FR-4.2:** The frontend dashboard shall render explainability visual outputs (such as feature impact rankings or summary lists) indicating which parameters (e.g., high $PM_{2.5}$ or lower humidity) drove the prediction outcome.

FR-5: Visualization and Reporting Module

- **FR-5.1:** The web dashboard shall display current sensor metrics and historical AQI trend graphs (daily, weekly, or monthly line charts).
- **FR-5.2:** The system shall provide an option to download a comprehensive analytical PDF summary report containing prediction history and explainability insights.

4. Non-Functional Requirements

4.1 Performance Requirements

- **FRC-4.1.1 (Response Time):** The web dashboard and interactive charts shall load and render within 2 to 3 seconds under normal network conditions.

- **FRC-4.1.2 (Inference Latency):** The backend machine learning prediction model and XAI (SHAP) calculation engine shall process input sensor parameters and return results within 1.5 seconds.
- **FRC-4.1.3 (Concurrency):** The Flask backend API shall support multiple concurrent user requests (at least 50 simultaneous users) without significant performance degradation or request throttling.

4.2 Reliability and Availability

- **FRC-4.2.1 (System Uptime):** The core web platform and backend API services shall maintain an operational availability of at least 99% during active monitoring periods.
- **FRC-4.2.2 (Error Handling):** The system shall implement robust exception-handling routines to gracefully manage corrupt CSV uploads, missing input fields, or failed database connection queries, displaying descriptive error messages to the user.

4.3 Security Requirements

- **FRC-4.3.1 (Data Transport Security):** All client-server communications between the React frontend and Flask backend shall be encrypted using HTTPS/TLS protocols.
- **FRC-4.3.2 (Authentication Security):** User passwords shall be securely hashed and salted (e.g., using bcrypt) prior to storage in the MySQL database.
- **FRC-4.3.3 (Access Control):** Role-Based Access Control (RBAC) shall be enforced to restrict administrative or data-management operations to authorized users only.

4.4 Usability Requirements

- **FRC-4.4.1 (Responsive Interface):** The React-based frontend UI shall be fully responsive, ensuring seamless viewing and interaction across desktops, laptops, tablets, and mobile devices.
- **FRC-4.4.2 (Accessibility):** Air quality categories (Good, Moderate, Unhealthy) shall be accompanied by standardized international color indicators and clear textual summaries to ensure comprehension by non-technical citizens.

4.5 Maintainability and Scalability

- **FRC-4.5.1 (Modular Architecture):** The backend code shall follow a decoupled, modular design pattern (separate scripts for preprocessing, prediction, explainability, and database routing) to ease future updates and model swapping.
- **FRC-4.5.2 (Database Scalability):** The MySQL database schema shall be properly indexed on timestamp and location parameters to ensure fast retrieval of historical logs as dataset sizes grow.

5. External Interface Requirements

5.1 User Interfaces (UI)

- **EI-5.1.1:** The platform shall provide a responsive, web-based graphical user interface (GUI) developed using **React.js**, ensuring a clean and modern user experience.

- **EI-5.1.2:** The interface shall support multi-device compatibility, adapting seamlessly across various screen resolutions including desktop monitors, laptops, tablets, and mobile smartphones.
- **EI-5.1.3:** The dashboard layout shall feature dedicated navigation elements (Home, Dashboard, Prediction, Analytics, Reports) for smooth transitions between system modules.
- **EI-5.1.4:** Data visualization components (such as AQI trend lines and SHAP feature importance graphs) shall utilize dynamic, interactive charts (e.g., Chart.js or Recharts) for clear readability.

5.2 Software Interfaces

- **EI-5.2.1: Backend REST API:** The React frontend shall communicate with the Python **Flask backend** via standard HTTP/REST methods (GET, POST) using JSON payloads for data exchange.
- **EI-5.2.2: Database Management System:** The application shall interface with a **MySQL database** for persistent storage and retrieval of user records, historical sensor datasets, and prediction audit logs.
- **EI-5.2.3: Machine Learning & XAI Libraries:** The backend shall integrate seamlessly with Python-based scientific libraries including **Scikit-Learn / XGBoost** for predictive analytics and **SHAP / LIME** for explainable AI computations.

5.3 Hardware Interfaces

- **EI-5.3.1:** The system shall require standard client-side hardware capable of running modern web browsers with an active internet connection.
- **EI-5.3.2:** The server hosting the Flask application and MySQL database shall possess adequate computational processing power and RAM to execute real-time machine learning inferences and SHAP calculations efficiently.

5.4 Communications Interfaces

- **EI-5.4.1:** All communication streams between the client browser, React frontend, and Flask server shall be secured using **HTTPS (Hypertext Transfer Protocol Secure)** over standard web ports.
- **EI-5.4.2:** Data interchange between tiers shall be structured strictly using standard **JSON (JavaScript Object Notation)** formats to ensure high interoperability.